



EDITORA AGÊNTICA

GUIA DE EXECUÇÃO

DETECTOR UNIVERSAL DE LLMs GRATUITAS — MAPEAMENTO DE HARNESS, PROVEDORES E MODELOS ATIVOS

NÍVEL INTERMEDIÁRIO

Heverton Eduardo Peres

ESPECIALISTA EM MARKETING E DESENVOLVIMENTO DE SOLUÇÕES

Heverton Eduardo Peres

GUIA DE EXECUÇÃO

**DETECTOR UNIVERSAL DE LLMs GRATUITAS — MAPEAMENTO DE
HARNESS, PROVEDORES E MODELOS ATIVOS**

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Sumário

GUIA DE EXECUÇÃO — DETECTOR UNIVERSAL DE LLMs GRATUITAS	4
1. O que é	4
2. Onde vive	4
3. Pré-requisitos	5
4. Instalação	5
4.1 Em projeto que já usa o submodule	5
4.2 Em projeto novo (sem submodule)	5
4.3 Execução direta (sem setup)	5
5. Uso	6
5.1 Relatório completo (com teste real)	6
5.2 Só mapear (sem rede, rápido)	6
5.3 Com timeout customizado	6
6. Saída esperada	6
6.1 Colunas da tabela principal	7
6.2 Seção “PROVEDORES PÚBLICOS DISPONÍVEIS”	8
7. Harnesses suportados	8
7.1 Prioridade de detecção do harness atual	8
7.2 Plataforma OMP	8
8. Provedores públicos mapeados	9
9. Como funciona internamente	10
9.1 Detecção de harness	10
9.2 Extração de provedores	10
9.3 Teste real	10
9.4 Classificação	10
10. Segurança	10
11. Solução de problemas	11
12. Referências rápidas	11

GUIA DE EXECUÇÃO — DETECTOR UNIVERSAL DE LLMs GRATUITAS

1. O que é

O **Detector Universal de LLMs Gratuitas** (`detectar_llms_gratuitas.py`) é uma ferramenta standalone (zero dependências externas) que:

1. **Detecta todos os harnesses** instalados na máquina (OMP, Claude Code, Grok, Gemini, Cursor, Windsurf, Codex, Continue, Antigravity, Kiro).
2. **Lê os provedores** configurados no harness atual (via `opencode.json`, `auth.json`, `env vars`).
3. **Lista os modelos gratuitos** disponíveis na plataforma (via `opencode models`).
4. **Testa cada modelo** e classifica como **ATIVO** (funciona) ou **INATIVO** (não funciona).
5. **Mostra provedores públicos** que o usuário ainda não configurou, com link direto para obter a chave.

Compatibilidade: funciona em qualquer projeto, em qualquer harness, sem configuração prévia.

2. Onde vive

Local	Caminho
Submodule (fonte canônica)	<code>.token-economy/ scripts/detectar_llms_gratuitas.py</code>
No projeto (via setup)	<code>scripts/token- economy/detectar_llms_gratuitas.py</code> (junction/symlink)

3. Pré-requisitos

- Python 3.9+ (stdlib apenas — `urllib`, `json`, `os`, `sys`, `pathlib`, `subprocess`)
- Conexão com a internet (para testes reais de LLM)
- Harness com provedores configurados (ou env vars de API keys)

Nenhuma instalação de pacotes extras é necessária.

4. Instalação

4.1 Em projeto que já usa o submodule

```
cd meu-projeto
powershell -ExecutionPolicy Bypass -File .token-economy\setup.ps1 #
Windows
bash .token-economy/setup.sh #
macOS/Linux
```

4.2 Em projeto novo (sem submodule)

```
cd meu-projeto-novo
git submodule add git@github.com:Heverton-web/token-economy-
shared.git .token-economy
powershell -ExecutionPolicy Bypass -File .token-economy\setup.ps1
```

4.3 Execução direta (sem setup)

```
python /caminho/para/.token-economy/scripts/detectar_llms_gratuitas.py
```

5. Uso

5.1 Relatório completo (com teste real)

```
python scripts/token-economy/detectar_llms_gratuitas.py
```

5.2 Só mapear (sem rede, rápido)

```
python scripts/token-economy/detectar_llms_gratuitas.py --sem-teste
```

5.3 Com timeout customizado

```
python scripts/token-economy/detectar_llms_gratuitas.py --timeout 15
```

6. Saída esperada

HARNESSES INSTALADOS: OMP, Claude Code, Grok, Gemini, Windsurf, Codex, Continue, Antigravity
HARNESS ATUAL: OMP
SESSÃO: C:\Users\...\proj_fabrica-de-livros

PROVEDOR	LLM	FREE
STATUS		

XIAOMI	mimo-v2.5-pro	● Não
● Ativo		
XIAOMI	mimo-v2.5	● Não
● Ativo		
OPENCODE	deepseek-v4-flash-free	● Sim
● Ativo		
OPENCODE	laguna-s-2.1-free	● Sim
● Ativo		
OPENROUTER	nvidia/nemotron-3-super-120b-a12b:free	● Sim
● Ativo		
ZENMUX	anthropic/claude-sonnet-5-free	● Sim

● Ativo

...

PROVEDORES PÚBLICOS DISPONÍVEIS (configure a chave para usar):

PROVEDOR	LLM	LINK
GOOGLE GEMINI	gemini-2.5-flash	https://aistudio.google.com/
GITHUB MODELS	gpt-4o-mini	https://github.com/marketplace/models
SAMBANOVA CLOUD	Meta-Llama-3.1-405B-Instruct	https://cloud.sambanova.ai/
CEREBRAS	llama3.1-8b	https://cloud.cerebras.ai/
HUGGING FACE	meta-llama/Llama-3.2-3B-Instruct	https://huggingface.co/settings/tokens
MISTRAL AI	open-mistral-7b	https://console.mistral.ai/
COHERE	command-r	https://dashboard.cohere.com/
DEEPSEEK	deepseek-chat	https://platform.deepseek.com/
XAI / GROK	grok-2-latest	https://console.x.ai/

6.1 Colunas da tabela principal

Coluna	Significado
PROVEDOR	Nome do provedor que fornece o modelo (NVIDIA, XIAOMI, OPENCODE, etc.)
LLM	Nome do modelo de linguagem
FREE	● Sim = gratuito / ● Não = pago
STATUS	● Ativo = funciona de verdade / ● Inativo = não funciona ou sem chave

6.2 Seção “PROVEDORES PÚBLICOS DISPONÍVEIS”

Mostra provedores gratuitos que o usuário **ainda não configurou** no harness atual. Cada linha inclui um link direto para obter a chave de API. Provedores já configurados no `auth.json` ou `opencode.json` são automaticamente excluídos desta seção.

7. Harnesses suportados

Harness	Deteção	Config lida
OMP / OpenCode / MiMo-Code / Orca	Env <code>OMPCODE=1</code>	<code>opencode.json</code> , <code>auth.json</code> , <code>oh-my-opencode-slim.json</code>
Claude Code	Env <code>CLAUDECODE=1</code>	<code>.claude/settings.json</code> , <code>.credentials.json</code>
Grok	<code>~/.grok/config.toml</code>	<code>config.toml</code> , <code>auth.json</code>
Gemini	<code>~/.gemini/settings.json</code>	<code>settings.json</code> , <code>google_accounts.json</code>
Cursor	<code>~/.cursor/settings.json</code>	<code>settings.json</code>
Windsurf	<code>~/.codeium/windsurf/</code>	<code>mcp_config.json</code>
Codex	<code>~/.codex/config.toml</code>	<code>config.toml</code>
Continue	<code>~/.continue/config.json</code>	<code>config.json</code>
Antigravity	<code>~/.antigravity/</code>	<code>argv.json</code>
Kiro	<code>~/.kiro/</code>	<code>config.json</code>

7.1 Prioridade de detecção do harness atual

1. Env vars do processo (`OMPCODE`, `CLAUDECODE`, `CODEX_HOME`, etc.)
2. Config no projeto (CWD)
3. Config na HOME

7.2 Plataforma OMP

O OMP roteia modelos de múltiplos provedores (NVIDIA, OpenRouter, ZenMux, etc.) via sua infraestrutura. Os modelos são detectados via `opencode models` e classificados como gratuitos

por: - Sufixo `-free` no nome do modelo - Sufixo `:free` no nome do modelo - Presença no preset zen do `oh-my-opencode-slim.json`

Credenciais: ficam em `~/.local/share/opencode/auth.json` (não no `opencode.json`).

8. Provedores públicos mapeados

Provedor	Endpoint	Modelos gratuitos	Link para chave
Google Gemini	<code>ai.language.google.com</code>	gemini-2.5-flash, gemini-2.0-flash, gemini-1.5-flash	https://aistudio.google.com/
Groq Cloud	<code>api.groq.com</code>	llama-3.3-70b-versatile, llama-3.1-8b-instant, gemma2-9b-it	https://console.groq.com/
OpenRouter	<code>openrouter.ai</code>	modelos <code>:free</code> (Gemini, Llama, Mistral, Qwen)	https://openrouter.ai/
GitHub Models	<code>models.inference.ai.azure.com</code>	gpt-4o-mini, llama-3.3-70b, Phi-3.5-mini	https://github.com/marketplace/models
SambaNova	<code>api.sambanova.ai</code>	Llama 3.1 405B/70B/8B	https://cloud.sambanova.ai/
Cerebras	<code>api.cerebras.ai</code>	llama3.1-8b, llama3.1-70b	https://cloud.cerebras.ai/
Hugging Face	<code>router.huggingface.co</code>	Llama 3.2 3B, Mistral 7B	https://huggingface.co/settings/tokens
Mistral AI	<code>api.mistral.ai</code>	open-mistral-7b, mistral-3b	https://console.mistral.ai/
Cohere	<code>api.cohere.com</code>	command-r, command-r-plus	https://dashboard.cohere.com/
DeepSeek	<code>api.deepseek.com</code>	deepseek-chat	https://platform.deepseek.com/
xAI / Grok	<code>api.x.ai</code>	grok-2-latest, grok-beta	https://console.x.ai/

9. Como funciona internamente

9.1 Detecção de harness

O script varre caminhos conhecidos de config de cada harness (HOME + CWD + env vars). O harness “atual” é o que tiver env var ativa.

9.2 Extração de provedores

- **OMP:** `opencode.json` (providers inline) + `auth.json` (credenciais) + `opencode models` (lista completa)
- **Claude Code:** `.credentials.json` (OAuth)
- **Outros:** `config.toml`, `settings.json`, etc.

9.3 Teste real

- **OMP:** verifica presença em `opencode models` (checagem local, sem rede)
- **Provedores HTTP:** chamada mínima `POST /chat/completions` com `max_tokens=1`

9.4 Classificação

Resultado	Classificação
Modelo listado em <code>opencode models</code>	● Ativo (plataforma)
HTTP 200	● Ativo (API)
HTTP 401/403/404/429	● Inativo (motivo indicado)
Sem chave	● Inativo (sem credencial)

10. Segurança

- **Nunca imprime chaves de API.** Apenas “presente” ou “ausente”.
- **Zero dependências externas** — stdlib apenas.
- **Execução local** — nenhuma telemetria enviada.

11. Solução de problemas

Problema	Causa	Solução
Harness não detectado	Config não encontrada	Verifique se o harness está instalado
Todos INATIVOS	Nenhuma chave configurada	Configure env vars ou adicione provider no harness
Erro de rede	Sem internet	Verifique conectividade; use <code>--sem-teste</code>
Timeout	Conexão lenta	<code>--timeout 30</code>
Modelo listado mas 404	Modelo indisponível na plataforma	OMP pode listar modelos que ainda não estão servindo

12. Referências rápidas

Comando	Função
<code>python scripts/token-economy/detectar_llms_gratuitas.py</code>	Relatório completo
<code>--sem-teste</code>	Só mapear, sem chamadas
<code>--timeout 15</code>	Timeout de 15s

Guia mantido por `scripts/atualizar-documentacao.py` — não edite o PDF à mão; edite este `.md` e recompile.

